

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442
Design for Higher

COURSE NAME: Compiler

Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error

recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4 Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I **V.VANDHANA VARSHA (192524265)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **V.VANDHANA VARSHA**

Course Faculty:**DEEPA.N**

Q1. Predictive Parser Design (Application Level)

Task:

Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

Grammar:

$E \rightarrow T E \mid \epsilon$

$E \mid \epsilon \rightarrow + T E \mid \epsilon$

$T \rightarrow F T \mid \epsilon$

$T \mid \epsilon \rightarrow * F T \mid \epsilon$

$F \rightarrow (E) \mid id$

Conditions:

- ☐ Remove left recursion (if any)
- ☐ Compute FIRST and FOLLOW sets
- ☐ Construct LL(1) parsing table

Test Case:

Input String:

id + id * id

Expected Output:

- ☐ Step-by-step parsing actions
- ☐ Final result: String Accepted

```
main.py
19 ("E", "id"): ["T", "E"],
20 ("E", "("): ["T", "E"],
21
22 ("E", "+"): ["+", "T", "E"],
23 ("E", ")"): ["e"],
24 ("E", "$"): ["e"],
25
26 ("T", "id"): ["F", "T"],
27 ("T", "("): ["F", "T"],
28
29 ("T", "*"): ["*", "F", "T"],
30 ("T", "+"): ["e"],
31 ("T", ")"): ["e"],
32 ("T", "$"): ["e"],
33
34 ("F", "id"): ["id"],
35 ("F", "("): ["(", "E", ")"]
36 }
37
38 input_string = ["id", "+", "id", "*", "id", "$"]
39
40 stack = ["$", "E"]
41
42 print("=" * 75)
43 print("LL(1) Predictive Parsing")
44 print("=" * 75)
```

Output

LL(1) Predictive Parsing

FIRST Sets

FIRST(E) = {'(', 'id'}
FIRST(E') = {'+', 'e'}
FIRST(T) = {'(', 'id'}
FIRST(T') = {'*', 'e'}
FIRST(F) = {'(', 'id'}

FOLLOW Sets

FOLLOW(E) = {'\$', ')'}
FOLLOW(E') = {'\$', ')'}
FOLLOW(T) = {'+', '\$', ')'}
FOLLOW(T') = {'+', '\$', ')'}
FOLLOW(F) = {'\$', '*', '+'}

Parsing Process

Stack	Input	Action
\$ E	id + id * id \$	E -> T E'
\$ E' T	id + id * id \$	T -> F T'
\$ E' T' F	id + id * id \$	F -> id
\$ E' T' id	id + id * id \$	Match id

Q2. Shift-Reduce Parsing (Application Level)

Task:

Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Conditions:

- ☐ Show stack, input buffer, and action at each step
- ☐ Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:

Input:

id + id * id

Expected Output:

- ☐ Sequence of actions: Shift / Reduce / Accept
- ☐ Final result: String Accepted

Integrated Design Question (SLR, LALR, CLR Parsing)

```
main.py  Run  Output  Clear

1
2
3 tokens = ["id", "+", "id", "*", "id", "$"]
4
5 stack = []
6
7 print("=" * 70)
8 print("SHIFT REDUCE PARSER")
9 print("=" * 70)
10
11 print("{:<25}{:<25}{:<25}".format("Stack", "Input", "Action"))
12 print("-" * 70)
13
14 i = 0
15
16 while i < len(tokens):
17
18     # SHIFT
19     stack.append(tokens[i])
20
21     print("{:<25}{:<25}Shift {}".format(
22         " ".join(stack),
23         " ".join(tokens[i+1:]),
24         tokens[i]
25     ))
26
```

```
=====
SHIFT REDUCE PARSER
=====
Stack          Input          Action
-----
id             + id * id $    Shift id
E              + id * id $    Reduce: E -> id
E +            id * id $    Shift +
E + id         * id $    Shift id
E + E          * id $    Reduce: E -> id
E              * id $    Reduce: E -> E + E
E *            id $    Shift *
E * id         $      Shift id
E * E          $      Reduce: E -> id
E              $      Reduce: E -> E * E
E $            $      Shift $
=====
String Accepted

=== Code Execution Successful ===
```

Q7. Operator Precedence Parsing Design and Evaluation

Task:

Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Conditions:

- ☐ Define operator precedence relations (<, =, >)
- ☐ Construct precedence table
- ☐ Handle associativity rules explicitly
- ☐ Detect and resolve conflicts
- ☐ Show step-by-step parsing actions
- ☐ Evaluate correctness of parsing decisions

Test Case:

Input: `id + id * id`

```
main.py  Run  Output  Clear
```

```
1
2
3 tokens = ["id", "+", "id", "*", "id", "$"]
4
5 stack = []
6
7 print("=" * 75)
8 print("OPERATOR PRECEDENCE PARSER")
9 print("=" * 75)
10
11 print("{:<30}{:<25}{:<25}".format("Stack", "Input", "Action"))
12 print("-" * 75)
13
14 i = 0
15
16 while i < len(tokens):
17
18     # Shift
19     stack.append(tokens[i])
20
21     print("{:<30}{:<25}Shift {}".format(
22         " ".join(stack),
23         " ".join(tokens[i+1:]),
24         tokens[i]
25     ))
26
```

```
=====
OPERATOR PRECEDENCE PARSER
=====
Stack          Input          Action
-----
id             + id * id $    Shift id
E              + id * id $    Reduce: E -> id
E +            id * id $    Shift +
E + id         * id $    Shift id
E + E          * id $    Reduce: E -> id
E              * id $    Reduce: E -> E + E
E *            id $    Shift *
E * id         $      Shift id
E * E          $      Reduce: E -> id
E              $      Reduce: E -> E * E
E $            $      Shift $
=====
String Accepted

Operator Precedence Used:
1. * has higher precedence than +
2. Both operators are left associative

=== Code Execution Successful ===
```

Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. Analyze how PEG parsers resolve ambiguity differently from CFG-based parsers
2. Compare PEG with LL and LR parsing in terms of:
 - o Grammar expressiveness
 - o Ease of implementation
 - o Performance and backtracking
3. Examine real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. Critically evaluate limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. Conclude whether PEG is suitable for all programming languages or only specific cases, with

Justification

